Hello new developer! Here's your guide to getting started with writing firmware for the VMS VCU system!

A couple orders of business before starting, here's the tools we're going to use:

- STM32C092RC boards STM32C092RC | Product - STMicroelectronics
- Zephyr RTOS Getting Started Guide — Zephyr Project Documentation
- Your choice IDE (VS Code always a favorite) Tutorial: Get started with Visual Studio Code
- CAN FD communication protocol Microchip University CAN

These tools were chosen for their versatility, price, featureset, and ease of development. It's recommended that you at least look at the C0 datasheet, learn how to set up Zepyhr dev tools and the flashing utility, get familiar with your IDE, and at least watch the CAN basics course in full if not the CAN development course before continuing. "Give me six hours to chop down a tree and I'll spend the first four sharpening the axe." - Lincoln, presumably.
The Zephyr getting started guide is a great starting point for setting up development tools, however to break into CAN the Zephyr CAN driver page is an excellent resource with CAN samples that can be built upon.

- Zephyr CAN page: Controller Area Network (CAN) — Zephyr Project Documentation
- CAN samples main: zephyr/samples/drivers/can at main · zephyrproject-rtos/zephyr · GitHub

In the samples ("samples" under the CAN controller title) there is a counter and a babbling example. Either of these examples are a good base for creating a CAN program that can filter and send messages on a bus.

Zephyr RTOS was partially chosen for the ability to use the CAN controller in real-time though the CAN shell.

- Zephyr Shell basics: https://dev.blues.io/blog/zephyr-use-shells-for-prototyping/
- Using Zephyr and Shell on Linux: https://michaelangerer.dev/zephyr/2023/02/12/zephyr-basics-shell.html
- Zephyr Shell Official Site: https://docs.zephyrproject.org/latest/services/shell/index.html
- CAN shell commands: https://docs.zephyrproject.org/latest/hardware/peripherals/can/shell.html

how

Reading Sensors

At this point you should already have gone through the Zephyr RTOS [Getting Started Guide — Zephyr Project Documentation]. If not, do this first and get blinky running on your board.

Next, download the appropriate documentation for the board you are using, below you will find the datasheets for the nucleo c092rc (MB2046)

Nucleo_c092rc board user manual:
https://www.st.com/resource/en/user_manual/um3353-stm32-nucleo64-board-mb2046-stmicroelectronics.pdf
Zephyr documentation for Nucleo_c092rc board:
https://docs.zephyrproject.org/latest/boards/st/nucleo_c092rc/doc/index.html

Potentiometer

We will set up the board to read from a potentiometer using one of the boards analog inputs paired with an adc sample. The following steps will get you started:

1.) Navigate to your zephyr project directory and find the adc_dt sample, this should be in <zephyrproject>/zephyr/samples/drivers/adc/adc_dt.

2.) Open the boards folder to see if there is an existing overlay for your board. If so, skip to step 4, if not we will make one.

3.) Create board Overlay: The board overlays help make zephyr more portable by allowing you to define key parameters for the board you are using without having to modify the main source code.

a.) First, copy a similar board overlay and rename it to match your board, I copied the nucleo_c071rb and renamed it to nucleo_c092rc

b.) Next, Open the overlay and find the section that defines the adc input channels which should look like this:

```
/ {
    zephyr,user {
        /* adjust channel number according to pinmux in board.dts */
        io-channels = <&adc1 0>, <&adc1 1>, <&adc1 4>;
    };
};
```

c.) find your boards dts file, in <zephyrproject>/zephyr/boards/ (for c092rc, it should be <zephyrproject>\zephyr\boards\st\nucleo_c092rc\nucleo_c092rc.dts)

The dts file defines hardware configuration for the board, including peripheral routing. We need to find what pins are connected to the adc by default.

d.) Here is a sample from the c092 .dts file:

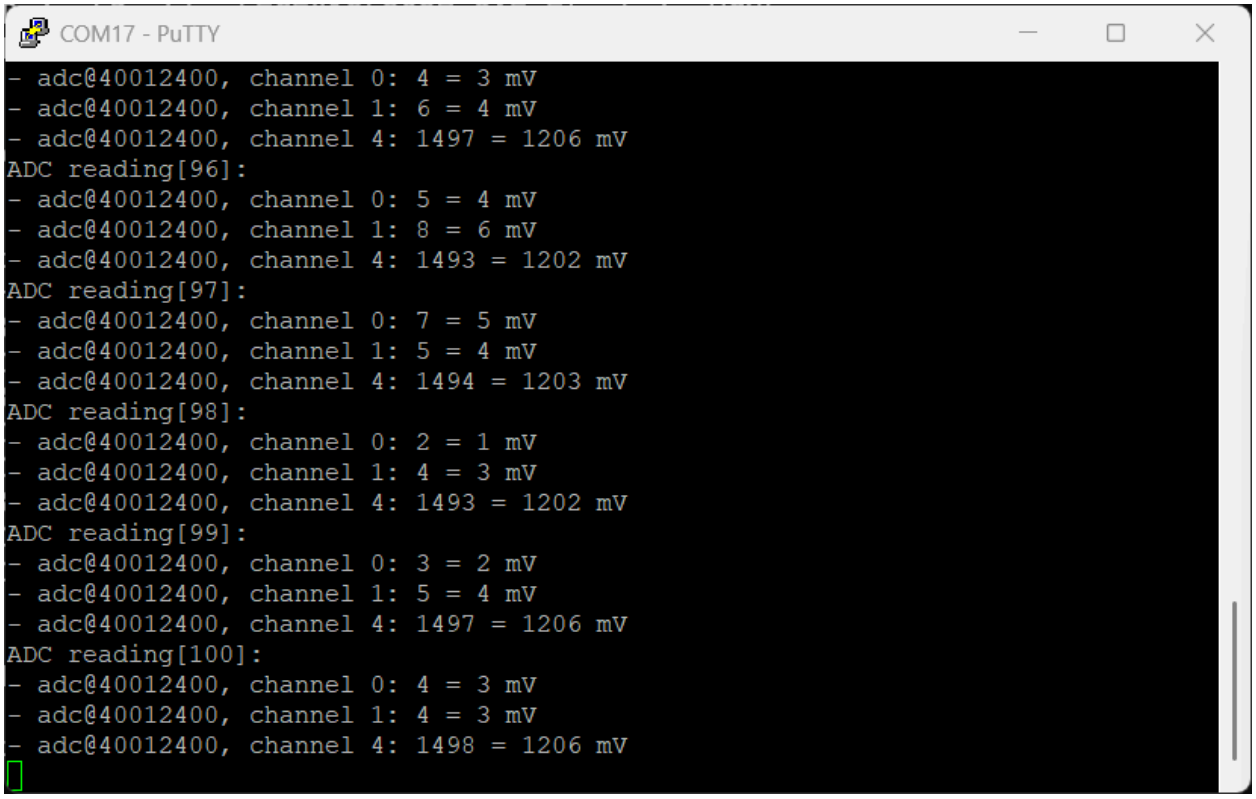
```
&adc1 {
    pinctrl-0 = <&adc1_in0_pa0 &adc1_in1_pa1 &adc1_in4_pa4>;
    pinctrl-names = "default";
    st,adc-clock-source = "ASYNC";
    clocks = <&rcc STM32_CLOCK(APB1_2, 20)>,
         <&rcc STM32_SRC_HSI ADC_SEL(2)>;
    clock-names = "adcx", "adc_ker";
    st,adc-prescaler = <4>;
    status = "okay";
    vref-mv = <3300>;
};
```

As you can see, the &adc1 peripheral is configured as an input to pa0, pa1, pa4
This matches the definitions in the c071 overlay which I copied, so no changes were needed. However, if you needed or wanted to change the input ports this is where you would do it.

4.) You now should have a board overlay for your board. If you haven't already, find and open your boards dts file in <zephyrproject>/zephyr/boards/ and find the adc peripheral section. Take note of the pin names that it is connected to. (for c092rc example above, pa0, pa1, pa4)

5.) Open the documentation for your board and search the pin names that you just found and note their location on the board. Also take note of the input range! (for c092rc, user-manual page 23, CN7/Pins 28,30,32)

6.) Finally, connect your sensor to one of the pins you identified, build and flash adc_dt sample, and use the serial monitor of your choice to monitor the output. You should see something like the image below


COM17 - PuTTY
- adc@40012400, channel 0: 4 = 3 mV
- adc@40012400, channel 1: 6 = 4 mV
- adc@40012400, channel 4: 1497 = 1206 mV
ADC reading[96]:
- adc@40012400, channel 0: 5 = 4 mV
- adc@40012400, channel 1: 8 = 6 mV
- adc@40012400, channel 4: 1493 = 1202 mV
ADC reading[97]:
- adc@40012400, channel 0: 7 = 5 mV
- adc@40012400, channel 1: 5 = 4 mV
- adc@40012400, channel 4: 1494 = 1203 mV
ADC reading[98]:
- adc@40012400, channel 0: 2 = 1 mV
- adc@40012400, channel 1: 4 = 3 mV
- adc@40012400, channel 4: 1493 = 1202 mV
ADC reading[99]:
- adc@40012400, channel 0: 3 = 2 mV
- adc@40012400, channel 1: 5 = 4 mV
- adc@40012400, channel 4: 1497 = 1206 mV
ADC reading[100]:
- adc@40012400, channel 0: 4 = 3 mV
- adc@40012400, channel 1: 4 = 3 mV
- adc@40012400, channel 4: 1498 = 1206 mV

ADC Input -> CAN Message

Before starting on this, you should have loaded and successfully ran both the adc and CAN counter samples on your board. Now we will combine these two samples into a new application that transmits input data onto the CAN bus.

1. First, create a copy of the can counter sample folder and rename it whatever you like. It should have the source code, boards folder, config files, etc.

2. Now we need to add all of the stuff in order to use ADC functionality in the application. This just means copying dissimilar things over from the adc_dt sample but here's a list of things to change:
    a. **Boards** folder: copy the overlay for your board from adc_dt to the new app
    b. **CMakeLists:** add ADC package to the list of required packages
    c. **Kconfig:** add ADC calibration requirement
    d. **prj.conf:** add 'CONFIG_ADC=y'

3. With the ADC functionality now enabled and configured for use in the new app, we can move on to the source code.

    Copy everything from the adc_dt main that is not already in our main file. There are some new header files and definitions, a function called adc_dt_spec, and all of the code in main to configure and read from the ADC.

    Within the main function, I put the ADC reading code inside the while(1) loop, after the change led section and before the counter message.

4. At this point, you should have an app that builds and runs but it will be printing the ADC values to the terminal directly instead of over CAN and it will still be sending the counter messages over the bus. Modify the main.c as follows:
    a. Comment out all of the print statements in the while 1 loop
    b. Move the can_send and UNALIGNED_PUT calls inside of the adc loop at the end
    c. Change the UNALIGNED_PUT command for the counter message so that it uses val_mv (adc_input) instead of 'counter'
    d. Scroll up and find the rx_thread function. Everything should be functional as it is, but you can change the print statements so they output more appropriate messages like 'adc value received' or whatever you like.

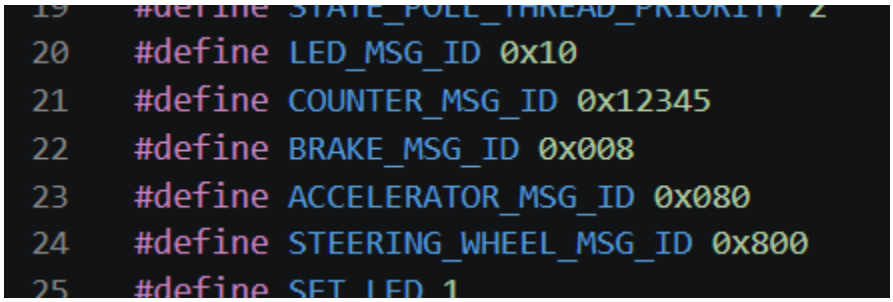
        This is the function where you filter messages and define what to do with those messages when you receive them.

5. Finally, build and flash to your board and your board should now read an adc input and output it onto the CAN bus

Receiving CAN Messages

In order to receive data we need to create a filter for the desired message id, add that id to the message queue, and define what to do with it once it is received.

First, we need to create the new message ID, you should see something like this at the top of an existing CAN app:



You can define the new message however you want, just remember that the message ID contains an implicit priority with lower IDs getting serviced first

Next, in the **rx_thread** function you should see an existing filter struct that looks like this

```
const struct can_filter brake_filter = {
    .flags = CAN_FILTER_IDE,
    .id = BRAKE_MSG_ID,
    .mask = CAN_STD_ID_MASK
};
```

TBD/Maybe not needed since we moved to Vulcan

Setting up vulcan and candlelight socketCAN

See the copperforge vulcan documentation here: https://copperforge.cc/docs/index.html
For initial setup of the vulcan.

Download the latest .bin file from the candlelight page
https://gitlab.com/Copperforge/Public/vulcan_firmware/vulcan_fw_candlelight/-/releases/

SocketCAN can only be run on a linux system, we have yet to figure out how to get it to work on windows.

On your linux system you will need to install can-utils and dfu-util which can be done be running the following commands

Sudo apt install dfu-util
Sudo apt install can-utils

Then just follow the copperforge setup guide to flash the candlelight socketCAN software to the vulcan

Using socketCAN device

Once you have flashed the vulcan with the software, you should be able to see the can device on your system to check. In the terminal, run:

ip link show

For more information, run

ip -details link show can0

This will tell you bitrate, samplepoint, and other configuration information on the device

This should show all of the network devices available and you should now have a device labeled “can0”

If this is your first time plugging it in, it will show “down” meaning it is inactive

To activate the can device run the command

sudo ip link set can0 up type can bitrate 500000

Now if you run **ip -detatils link show can0** again, you should see that the bus is “up” and bitrate is set for 500000.

You should also see that the status is “ERROR-ACTIVE” which means the CAN bus is working properly.

You can now run **candump can0** in one terminal window while you send can messages in a separate window using **cansend can0 <MSG_ID>#<PAYLOAD>**

For example,
Cansend can0 123#deadbeef

Which should display a message with ID 123 and data bytes DE AD BE EF in the candump window.

If it any time messages are not making it through, check to see if the bus status is “ERROR-PASSIVE” using the ip -details link command. This means that the device has received too many error messages and has errored out.

You can reset the bus by running
sudo ip link set can0 down
Then
sudo ip link set can0 up

You now have a functioning CAN network device, yay!

Configuring auto initialize for socketCAN device

To have the CAN bus automatically initialize we added a systemd-networkd file to tell the linux system to bring the bus up when it is recognized. This works for hotplugging the vulcan too!

In the VCU GitHub, find the configuration file labeled “80-can.network”

Copy this file to your system network files by running the following

sudo cp 80-can.network /etc/systemd/network/

Then run the command

sudo systemctl restart systemd-networkd

Your can device should now be activated and your linux system should automatically bring the CAN bus up on startup or hotplug.

Confirm that the bus is active and working by ensuring that it says “UP” and “ERROR-ACTIVE” when you run **ip -details link show can0**

Using SocketCan on Windows

This is kind of a pain and not at all necessary but if you are determined to develop on a windows system like I was then it isn't too hard to get set up.

You should probably know a little bit about building a Linux kernel before attempting but the guide below is pretty easy to follow regardless.

https://gist.github.com/manavortex/cdaf9540784808e5848cbec744d49a19

I had a couple issues when doing this:

1.) I was missing a tool required for building the kernel (it will throw an error during building and tell you which module is missing)
2.) The CAN device drivers menu was on a different page than what they show in the guide. (for me, it was in device drivers -> networking drivers -> CAN drivers, or something like that)
3.) You will also want to enable the usb-CAN driver that your device uses, for candlelight socketCAN on the vulcan, it is gs_usb (Geschwister Schneider)

Other than that, you will need to use the windows **usbipd** tools to attach usb devices to WSL

Usbipd list (to find bus id)
Usbipd bind -b <bus-id>
Usbipd -w -b <bus-id>

Then if you should see the device in WSL using lsusb and ip link show.

Writing a New Module using VCUmodule in Zephyr

This application is designed to split up all of the functionality needed for the entire car into separate .c/.h files so that you can only include what is necessary for a specific module. Each API contains essential functions for setting up and using that specific feature.

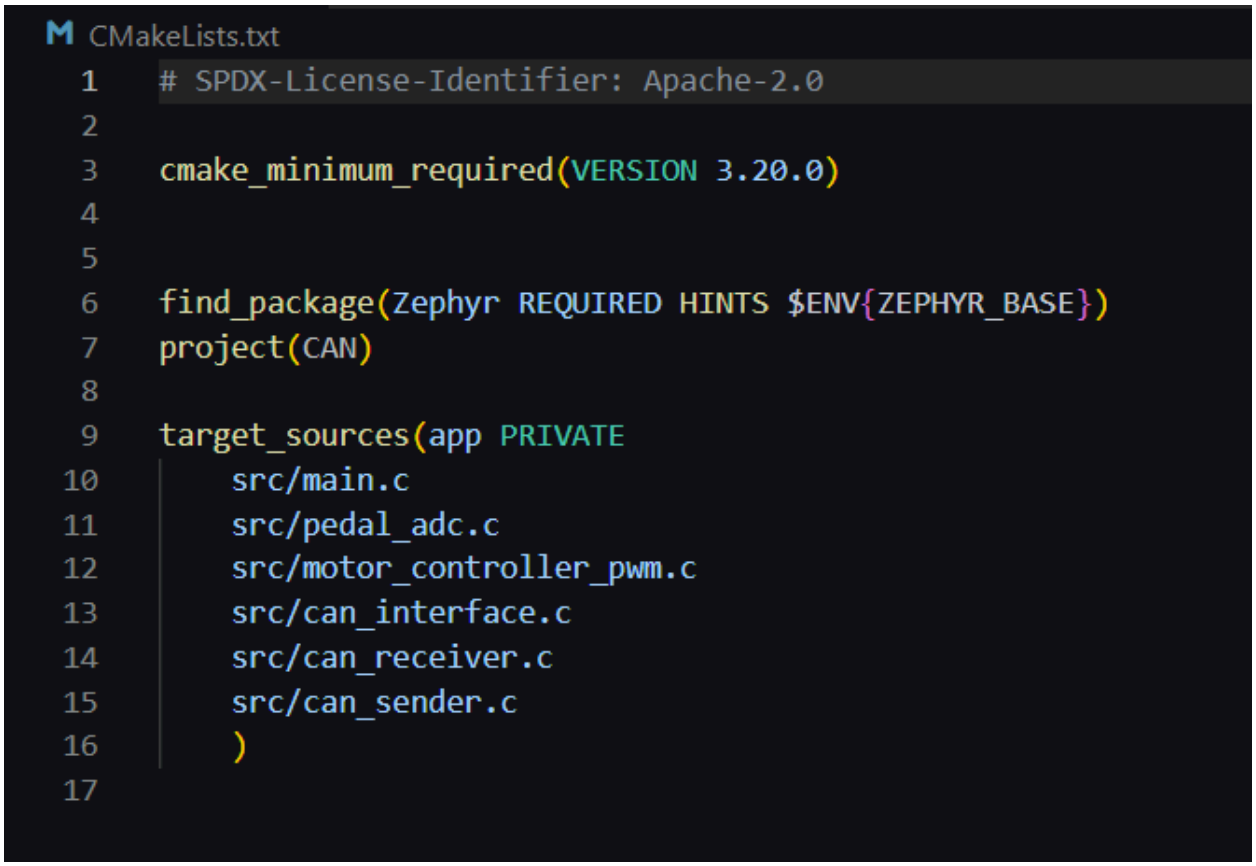
More information on specific features can be found in the .h and .c files

See the following outline for writing a new module.

1.) Copy VCUmodule to your zephyr workspace

2.) At the time of writing this, the nucleo_c092rc is the only board that we have written an overlay for. If you are using a different board, you will need to create a new one specific to your board that supports the functionality desired.

3.) Open the CMakeLists.txt file, you should see something like this:



This is where you will tell CMake which .c files you will be using for your module. You can delete the ones not needed for your module.
Similarly, you can add/remove the .h files from main as needed

```
#include "pedal_adc.h"
#include "motor_controller_pwm.h"
#include "can_interface.h"
#include "can_receiver.h"
#include "can_sender.h"
#include "can_database.h"
```

4.) Now you are ready to write main and add any functionality needed for the new module.

The main.c that you started with should serve as a good example of how to interact with the various APIs but each one also has fairly good documentation inside for where to add stuff.

5.) CAN Functionality

It should be fairly straightforward getting CAN functionality added with the APIs as they are well commented, but I'll explain each file briefly since there are a few.

can_database.h
This file simply contains the CAN message IDs and and comments about how they are used

can_interface
This contains all generic CAN initialization and is needed if CAN functionality will be utilized in the module

can_receiver
Contains functions for receiving data over the CAN bus. This is only needed if you need to receive messages from the bus.

Inside you will have to define filters for any message IDs that you add in order to receive them

Optionally, you can define what to do when specific messages are received here as well

can_sender
Contains functions for sending data over the CAN bus. This is only needed if you are sending data over the bus.

You should define CAN frames for the MSG IDs you are sending here. The sending function will default to 8 byte payload, std 11-bit ID, etc. if not defined.